Project - Low Level Design

on

Dockerized Healthcare Python Flask Service

Course Name : DevOps

Institution Name : Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s) :

Sr no	Student Name	Enrollment Number
1	Animesh Bokil	EN22CS301125
2	Anshika Vyas	EN22CS301154
3	Aryan Singh	EN22CS301215
4	Ashlesha Verma	EN22CS301219
5	Avani Rawal	EN22CS301238

Group Name: 10D2

Project Number: DO-22

Industry Mentor Name:

University Mentor Name: Prof. Akshay Saxena

Academic Year: 2022-2026

1. Introduction

1.1 Scope of the document
1.2
This document defines the detailed design and implementation of a CI/CD pipeline **Healthcare Python Flask Service**. It includes:

- Continuous Integration (CI) pipeline (build, test, linting)
- Continuous Deployment (CD) pipeline (Docker build & deployment)
- Multi-stage Dockerfile design
- Docker Compose setup for local development
- Kubernetes deployment configuration
- Automation workflows using GitHub Actions/Jenkins

1.2 Intended Audience

This document is intended for:

- **DevOps Engineers** – Pipeline setup and monitoring
- **Developers** – Understanding build and deployment
- **System Administrators** – Infrastructure management
- **QA Engineers** – Testing integration
- **Project Managers** – Workflow visibility

1.3 System Overview

This system automates building, testing, containerizing, and deploying a **Flask-based healthcare service**.

Key Components

- **Git Repository (GitHub)** – Source code management
- **CI/CD Tool (GitHub Actions / Jenkins)** – Pipeline automation
- **Docker** – Application containerization
- **Docker Compose** – Multi-container local setup
- **Container Registry (Docker Hub)** – Image storage
- **Kubernetes Cluster** – Deployment and orchestration
- **Database (PostgreSQL/MySQL)** – Patient data storage

Workflow Summary:

1. Developer pushes code to GitHub
2. CI pipeline triggers automatically

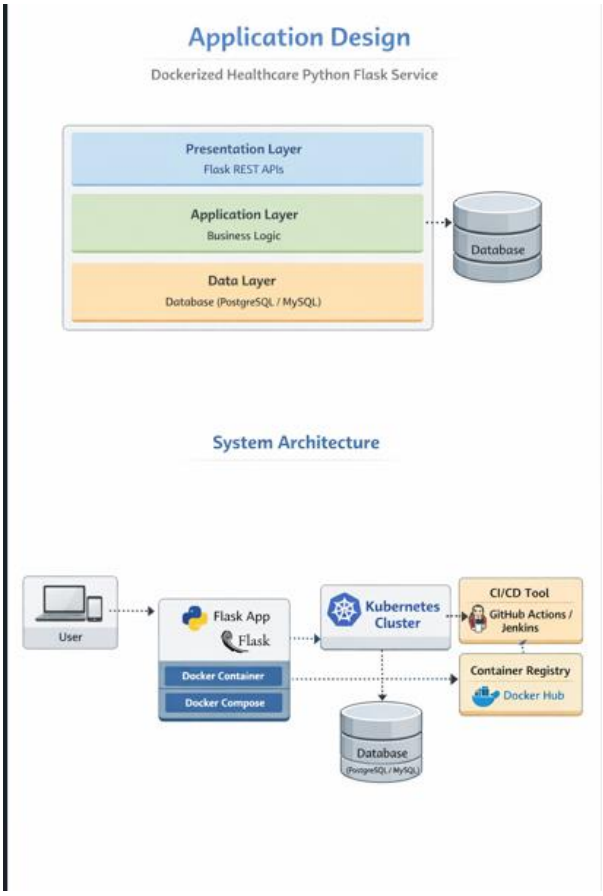
3. Code is tested and validated
4. Docker image is built using multi-stage Dockerfile
5. Image is pushed to Docker Hub
6. CD pipeline deploys to Kubernetes
7. Application becomes live

Key Features:

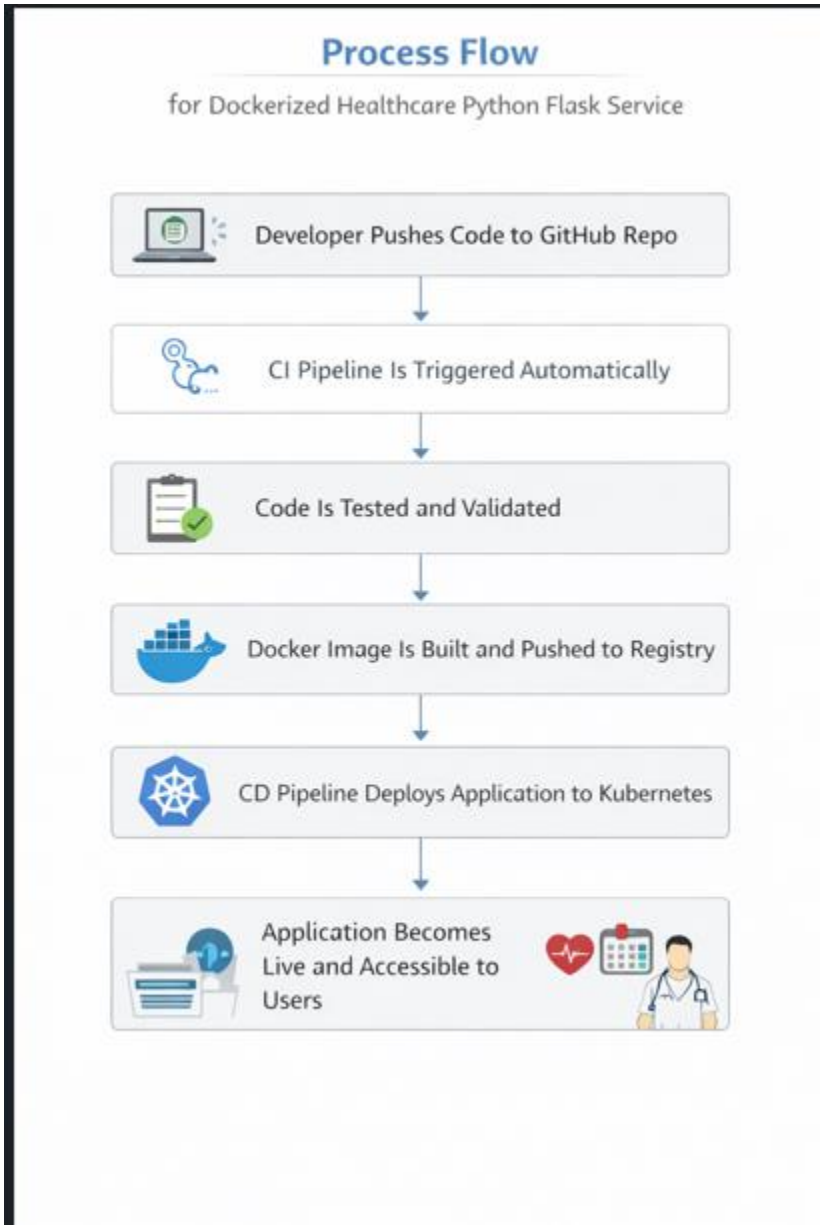
- Automated CI/CD pipeline
- Lightweight and secure Docker images
- Scalable Kubernetes deployment
- Environment consistency
- Reduced manual effort

2. System Design

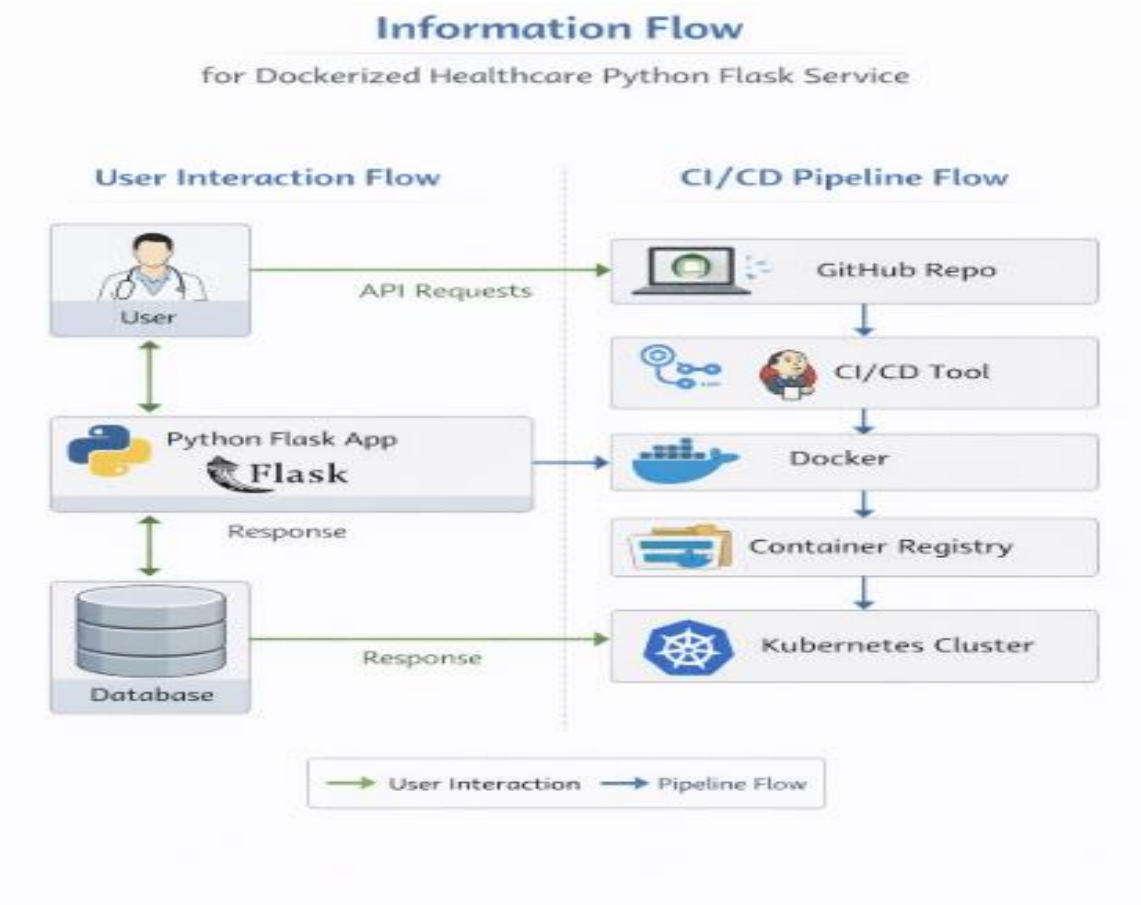
2.1 Application Design



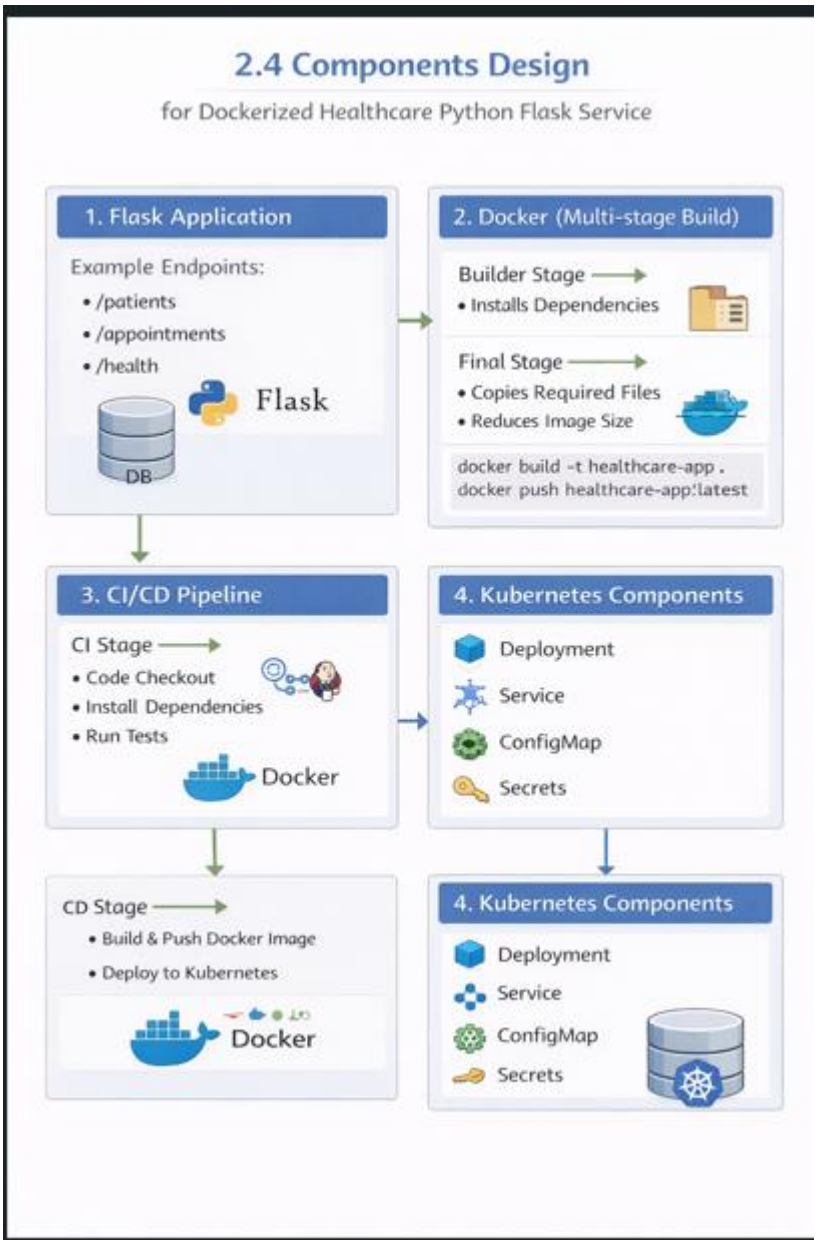
2.2 Process Flow



2.3 Information Flow



2.4 Components Design



2.5 Key Design Considerations

2.6 API Catalogue

Endpoint	Method	Description	Request Parameters	Response	Authentication
/api/health	GET	Check service status		JSON status (UP/DOWN)	No
/api/patients	GET	Get list of patients		None	Yes
/api/patients/id)	GET	Get patient details		Optional: page, limit	Yes
/api/patients	POST	Add new patient		id (path)	Yes
/api/patients	POST	name, age, gender, disease (body)		JSON patient details or 404	Yes
/api/patients/id)	PUT	id (path), updated fields (body)		JSON success message	Yes
/api/patients/id)	DELETE	id (path)		JSON updated patient	Yes
/api/appointments	GET	Optional: date, patient_id		JSON list of appointments	Yes
/api/appointments	GET	Get all appointments		JSON list of appointments	Yes
/api/appointments	POST	patient_id, date, doctor (body)		JSON created appointment	Yes
/api/appointments	DELETE	id (path)		JSON success message	Yes
/api/appointments /id)	DELETE	Cancel appointment		JSON success message	Yes

Yes: Requires valid authentication token

3. References

- Flask Documentation
- Docker Documentation
- Kubernetes Documentation
- GitHub Actions Documentation
- Jenkins Documentation
- Git Documentation
- Docker Hub Documentation